



Laços, funções e módulos

Coding: Linguagens e Técnicas · Aula 4 · 1 de setembro de 2026

Prof. Sebastião Rogério

Agenda

01

Laços

for, while, range e enumerate, mais a armadilha de alterar a lista enquanto a percorre.

02

Funções

Parâmetro, retorno, argumento padrão e escopo. Duas pegadinhas que derrubam sistema.

03

Módulos

Tirar tudo de um arquivo só e entender por que existe a linha do dunder main.

04

Teste automatizado

pytest, o teste que falha de propósito e a correção do defeito da Aula 3.



Ficou um defeito em aberto na aula passada. A comanda cobrou três serviços e a tela informou quatro itens.

Hoje esse defeito é corrigido, e o mais importante: passa a existir um teste que impede o retorno dele.

01

Laços

Repetir sem repetir código.

for percorre a coleção, não o índice

Em Java ou C você controla um contador. Em Python o laço entrega o item direto.

```
lacos.py

servicos = ["corte", "barba", "sobancelha"]

for servico in servicos:
    print(servico)
```

SAÍDA

corte

barba

sobancelha

E quando eu preciso mesmo do número? `range(5)` imprime até quanto?

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the text 'python'. The terminal displays two lines of Python code: a for loop that iterates over the range(5) and prints each value of i, with the end parameter set to a space character.

```
python  
  
for i in range(5):  
    print(i, end=" ")
```

Conte quantos números aparecem e qual é o último.

Cinco números, de 0 a 4. O limite não entra, igual ao fatiamento da aula passada.

python

```
for i in range(5):  
    print(i, end=" ")  
  
# range(inicio, fim) e range(inicio, fim, passo)  
print(list(range(1, 6)))  
print(list(range(0, 10, 2)))
```

SAÍDA

0 1 2 3 4

[1, 2, 3, 4, 5]

[0, 2, 4, 6, 8]

A mesma regra do fatiamento vale aqui: o começo entra, o fim não. Isso faz `range(len(lista))` percorrer exatamente os índices válidos.

Quando você precisa do item e da posição

enumerate devolve os dois de uma vez, e evita o contador manual.

```
lacos.py

servicos = ["corte", "barba", "sobrancelha"]

for i, servico in enumerate(servicos, start=1):
    print(i, servico)
```

SAÍDA

```
1 corte
2 barba
3 sobrancelha
```


Este código deveria deixar só corte e barba na lista. Ele consegue?

```
python

itens = ["corte", "pintura", "tintura", "barba"]

for item in itens:
    if item not in ("corte", "barba"):
        itens.remove(item)

print(itens)
```

O laço está percorrendo a mesma lista que está sendo alterada.

Não. Sobra tintura, e nenhum erro é exibido.

```
python  
  
itens = ["corte", "pintura", "tintura", "barba"]  
  
for item in itens:  
    if item not in ("corte", "barba"):  
        itens.remove(item)  
  
print(itens)
```

SAÍDA

```
['corte', 'tintura', 'barba']
```

Ao remover pintura da posição 1, tintura desce para a posição 1, mas o laço já avançou para a posição 2. O item pulado nunca é examinado.

A regra: nunca altere a coleção que está sendo percorrida

Monte uma lista nova. Custa uma linha a mais e elimina a classe inteira de defeito.

```
lacos.py

itens = ["corte", "pintura", "tintura", "barba"]

limpos = []
for item in itens:
    if item in ("corte", "barba"):
        limpos.append(item)

print(limpos)
```

SAÍDA

```
['corte', 'barba']
```

while: repete enquanto a condição for verdadeira

Use quando você não sabe de antemão quantas voltas vai dar.

```
lacos.py

saldo = 100
dias = 0

while saldo >= 35:
    saldo -= 35
    dias += 1

print(dias, saldo)
```

SAÍDA

2 30

for ou while

A pergunta é uma só: você sabe quantas voltas vai dar?

Situação	Use	Por quê
Percorrer uma lista inteira	for	A quantidade é o tamanho da coleção
Repetir um número fixo de vezes	for	range já define o limite
Esperar o usuário digitar algo válido	while	Pode acontecer na primeira ou na décima tentativa
Consumir saldo até acabar	while	Depende do valor, não da quantidade
Ler um arquivo até o fim	for	A própria linguagem sabe onde termina

Todo while precisa de uma linha que aproxime o fim. Sem ela, o programa trava e não avisa.

break sai do laço, continue pula a volta

continue é o que vai aparecer na comanda daqui a pouco.

```
lacos.py

for item in ["corte", "pintura", "barba"]:
    if item == "pintura":
        print("fora do catalogo, ignorando")
        continue
    print("cobrando", item)
```

SAÍDA

```
cobrando corte
fora do catalogo, ignorando
cobrando barba
```

Prática 1

10 min

Digite e rode. Cada item é um laço.

- 0 Percorra a lista de serviços da sua equipe e imprima cada um.
- 1 Imprima a mesma lista numerada a partir de 1, usando enumerate.
- 2 Some os preços de todos os itens de uma comanda com um for.
- 3 Usando while, descubra quantos cortes cabem em R\$ 200.
- 4 Percorra a comanda e imprima apenas os itens que estão no catálogo, usando continue.

02

Funções

Dar nome a um pedaço de código para poder testá-lo e reutilizá-lo.

def, parâmetro e retorno

O parâmetro é o nome na definição. O argumento é o valor na chamada.

```
funcoes.py

def preco_com_desconto(preco, percentual):
    return preco - (preco * percentual / 100)

print(preco_com_desconto(35, 10))
print(preco_com_desconto(100, 25))
```

SAÍDA

31.5

75.0

Esta função não tem return. O que a variável x recebe?

```
python

def mostrar(nome):
    print(f"Serviço: {nome}")

x = mostrar("corte")
print(x)
print(type(x))
```

Cuidado com a diferença entre imprimir e devolver.

Recebe None. Toda função sem return devolve None.

python

```
x = mostrar("corte")  
print(x)  
print(type(x))
```

SAÍDA

Servico: corte

None

<class 'NoneType'>

Imprimir mostra na tela e some. Devolver entrega o valor para quem chamou. Trocar um pelo outro é a causa mais comum de função que "não funciona" sem dar erro.

Argumento nomeado e valor padrão

O padrão vale quando o argumento não é informado, e permite chamar a função de forma mais curta.

```
funcoes.py

def preco_com_desconto(preco, percentual=10):
    return preco - (preco * percentual / 100)

print(preco_com_desconto(35))
print(preco_com_desconto(35, 20))
print(preco_com_desconto(preco=35, percentual=50))
```

SAÍDA

31.5

28.0

17.5

Cada chamada deveria começar uma comanda nova. Começa?

```
python

def registrar(item, comanda=[]):
    comanda.append(item)
    return comanda

print(registrar("corte"))
print(registrar("barba"))
print(registrar("sobancelha"))
```

A lista vazia do padrão é criada quantas vezes?

Não. A lista do padrão é criada uma única vez, quando a função é definida.

```
python  
  
print(registrar("corte"))  
print(registrar("barba"))  
print(registrar("sobancelha"))
```

SAÍDA

```
['corte']  
['corte', 'barba']  
['corte', 'barba', 'sobancelha']
```

Todas as chamadas compartilham o mesmo objeto, exatamente como as duas listas da aula passada. Em um sistema real, isso mistura a comanda de um cliente com a do próximo.

A correção padrão para esse caso

Use None como marcador e crie a lista dentro da função.

```
funcoes.py

def registrar(item, comanda=None):
    if comanda is None:
        comanda = []
    comanda.append(item)
    return comanda

print(registrar("corte"))
print(registrar("barba"))
```

SAÍDA

```
['corte']
['barba']
```



Valor padrão só pode ser algo que não muda: número, texto, tupla ou None. Nunca lista, dicionário ou conjunto.

Python Docs, FAQ de programação: por que valores padrão são compartilhados entre chamadas

A função tenta somar na variável de fora. Funciona?

```
python

total = 0

def somar(valor):
    total = total + valor
    return total

print(somar(35))
```

A variável total aparece dos dois lados do sinal de igual, dentro da função.

Quebra. Ao atribuir a `total` dentro da função, ela vira uma variável local.

python

```
def somar(valor):  
    total = total + valor    # local, ainda sem valor  
    return total
```

SAÍDA

Traceback (most recent call last):
UnboundLocalError

A saída correta é não depender de variável de fora. Receba o valor por parâmetro e devolva o resultado: assim a função pode ser testada isoladamente.

A mesma soma, sem depender de nada externo

Entra pelo parâmetro, sai pelo return. Essa é a forma que dá para testar.

```
funcoes.py

def somar(total, valor):
    return total + valor

total = 0
total = somar(total, 35)
total = somar(total, 25)

print(total)
```

SAÍDA

60

Sinais de que a função precisa ser dividida

Nenhum deles é regra absoluta. Todos são motivo para olhar de novo.

Sinal	O que costuma indicar
O nome tem "e" no meio	Faz duas coisas, e por isso é difícil de nomear
Passa de 20 linhas	Cabe extrair pelo menos um pedaço com nome próprio
Imprime e calcula ao mesmo tempo	Quem calcula não deveria decidir como mostrar
Recebe mais de 4 parâmetros	Talvez o conjunto deles seja um objeto ainda sem nome
Você não sabe explicar o retorno em uma frase	O contrato da função ainda não está claro

Prática 2: extrair funções da comanda

20 min

Pegue o arquivo `pratica5_comanda.py` da aula passada e transforme em funções.

- 0 `itens_validos(comanda)` devolve só os itens que existem no catálogo.
- 1 `subtotal(comanda)` devolve a soma dos itens válidos, em Decimal.
- 2 `desconto(comanda, valor)` devolve o desconto conforme a regra da sua equipe.
- 3 `fechar(comanda)` usa as três anteriores e devolve um dicionário com subtotal, desconto e total.
- 4 Nenhuma delas pode ter print dentro. Quem imprime é quem chama.
- 5 Corrija aqui o defeito da aula passada: o desconto deve olhar só para os itens válidos.

03

Módulos

Tirar tudo de um arquivo só, e a linha estranha que aparece no fim de todo script.

A estrutura de hoje

Dois arquivos na mesma pasta. O de teste sempre começa com test_.

```
terminal
ads-2026-2-barbearia/
└─ aula04/
    └─ comanda.py          # as funcoes
    └─ test_comanda.py     # os testes das funcoes
```

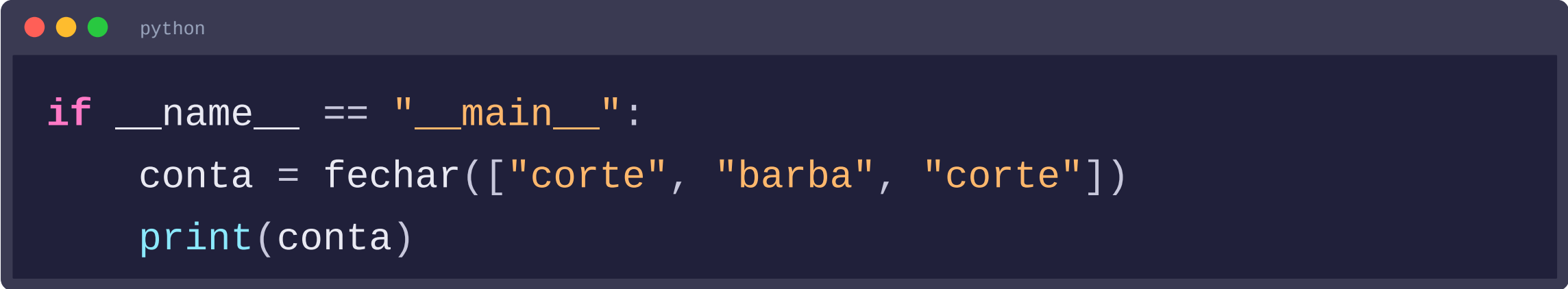
Importar do próprio projeto

O nome do módulo é o nome do arquivo, sem a extensão.

test_comanda.py

```
# traz apenas o que vai ser usado  
from comanda import fechar, itens_validos, subtotal  
  
# alternativa: traz o modulo inteiro  
# import comanda  
# comanda.fechar(...)
```


Toda apostila manda escrever esta linha no fim do arquivo. Para que ela serve?

A screenshot of a Python terminal window with a dark blue background. The window has three colored window control buttons (red, yellow, green) in the top-left corner. The text inside the terminal is as follows:

```
python  
  
if __name__ == "__main__":  
    conta = fechar(["corte", "barba", "corte"])  
    print(conta)
```

O que aconteceria com esse print se outro arquivo importasse este módulo?

Ela separa rodar o arquivo de importar o arquivo.

python

```
python comanda.py
```

```
__name__ vale "__main__"    -> o bloco EXECUTA
```

```
from comanda import fechar
```

```
__name__ vale "comanda"    -> o bloco NAO executa
```

Sem essa linha, importar o módulo dispararia o print. E é exatamente isso que o pytest faz ao importar o seu arquivo para testar.

O módulo pronto, parte 1

O catálogo vira constante no topo. A primeira função separa o que existe do que não existe.

```
comanda.py

from decimal import Decimal

PRECOS = {
    "corte": Decimal("35.00"),
    "barba": Decimal("25.00"),
    "sobrancelha": Decimal("15.00"),
    "hidratacao": Decimal("40.00"),
}

def itens_validos(comanda, precos=PRECOS):
    validos = []
    for item in comanda:
        if item in precos:
            validos.append(item)
    return validos
```

O módulo pronto, parte 2

A correção do defeito da Aula 3 está na condição do desconto.

```
comanda.py

def subtotal(comanda, precos=PRECOS):
    total = Decimal("0.00")
    for item in itens_validos(comanda, precos):
        total += precos[item]
    return total

def desconto(comanda, valor, minimo=3):
    if len(itens_validos(comanda)) >= minimo:
        return valor * Decimal("0.10")
    return Decimal("0.00")
```

O módulo pronto, parte 3

Nenhuma função imprime. O bloco final só existe para você rodar direto e olhar.

```
comanda.py

def fechar(comanda):
    sub = subtotal(comanda)
    desc = desconto(comanda, sub)
    return {"subtotal": sub, "desconto": desc, "total": sub - desc}

if __name__ == "__main__":
    conta = fechar(["corte", "barba", "corte", "pintura"])
    for chave, valor in conta.items():
        print(f"{chave:<9} R$ {valor:.2f}")
```

SAÍDA

```
subtotal  R$ 95.00
desconto  R$ 9.50
total     R$ 85.50
```

04

Teste automatizado

A parte que transforma "acho que está certo" em "está, e continua estando".



Um teste é a regra de negócio escrita de um jeito que o computador consegue conferir sozinho, toda vez.

Instalar e rodar

Uma linha para instalar, uma para rodar. O pytest encontra os arquivos sozinho.



```
terminal  
  
pip install pytest  
  
cd aula04  
python -m pytest -q
```


O primeiro teste

Uma função que começa com test_ e uma linha de assert. Só isso.

```
test_comanda.py

from decimal import Decimal

from comanda import fechar, itens_validos, subtotal

def test_subtotal_soma_apenas_o_que_esta_no_catalogo():
    assert subtotal(["corte", "barba", "pintura"]) == Decimal("60.00")
```

SAÍDA

1 passed in 0.01s

A anatomia de um teste

Três passos, sempre nessa ordem. Vale para qualquer linguagem.

Passo	O que faz	No exemplo
Preparar	Monta a situação de partida	a comanda com três itens
Executar	Chama a função sendo testada	subtotal(comanda)
Verificar	Compara o obtido com o esperado	assert ... == Decimal("60.00")

O nome do teste é documentação. `test_subtotal_soma_apenas_o_que_esta_no_catalogo` diz a regra sem precisar ler o corpo.

Este teste cobre o defeito da Aula 3. Ele passa no código de lá?

```
python
def test_item_fora_do_catalogo_nao_conta_para_o_desconto():
    conta = fechar(["corte", "barba", "pintura"])
    assert conta["desconto"] == Decimal("0.00")
```

Na Aula 3 o desconto olhava para len(comanda). Quantos itens tem essa comanda?

Falha. E a mensagem entrega o defeito de bandeja.

terminal

```
python -m pytest -q
```

SAÍDA

```
.F... [100%]  
> assert conta["desconto"] == Decimal("0.00")  
E      AssertionError: assert Decimal('6.0000') == Decimal('0.00')  
1 failed, 4 passed in 0.02s
```

A comanda tem três itens, mas só dois são cobrados. O desconto de 10% incidiu sobre R\$ 60 e resultou em R\$ 6. Era exatamente o defeito que a turma encontrou olhando a tela.

A correção, e o verde de volta

Uma palavra muda na condição. O teste garante que ela não volte atrás.

terminal

```
# antes: if len(comanda) >= minimo:  
# depois: if len(itens_validos(comanda)) >= minimo:
```

```
python -m pytest -q
```

SAÍDA

.....

[100%]

5 passed in 0.01s

A suíte completa

Cinco testes, cinco regras de negócio executáveis. No arquivo real, deixe duas linhas em branco entre as funções.

```
test_comanda.py

def test_subtotal_soma_apenas_o_que_esta_no_catalogo():
    assert subtotal(["corte", "barba", "pintura"]) == Decimal("60.00")

def test_item_fora_do_catalogo_nao_conta_para_o_desconto():
    conta = fechar(["corte", "barba", "pintura"])
    assert conta["desconto"] == Decimal("0.00")

def test_desconto_de_dez_por_cento_a_partir_de_tres_itens():
    conta = fechar(["corte", "barba", "corte"])
    assert conta["subtotal"] == Decimal("95.00")
    assert conta["desconto"] == Decimal("9.50")
    assert conta["total"] == Decimal("85.50")

def test_comanda_vazia_nao_quebra():
    assert fechar([])["total"] == Decimal("0.00")

def test_itens_validos_ignora_desconhecido():
    assert itens_validos(["corte", "pintura"]) == ["corte"]
```



Repare no último teste: a comanda vazia. É o caso que ninguém lembra de conferir na mão e que aparece no primeiro dia de uso real.

Prática 3: testar as suas funções

20 min

Sobre as funções que você extraiu na Prática 2, no domínio da sua equipe.

- 0 Crie `test_comanda.py` na mesma pasta e importe as suas funções.
- 1 Escreva um teste do caminho normal: uma comanda válida com o total esperado.
- 2 Escreva um teste do item fora do catálogo.
- 3 Escreva um teste da regra chata da sua equipe, no valor exato da fronteira.
- 4 Escreva um teste da coleção vazia.
- 5 Rode a suíte e só continue quando as cinco linhas estiverem verdes.

05

Fechar no Git

Mesma rotina da aula passada, agora com uma pasta a mais.

Os arquivos de hoje

Dois arquivos. O que o pytest cria não vai junto.

terminal

```
git status
```

```
git add aula04/comanda.py
```

```
git add aula04/test_comanda.py
```

```
# NAO adicione:
```

```
# .pytest_cache/
```

```
# __pycache__/
```

Commit e push

A frase diz o que o commit faz com o projeto.

terminal

```
git commit -m "aula04: extrai funcoes da comanda e adiciona testes"
```

```
git push origin main
```

SAÍDA

```
[main 9c4b2f1] aula04: extrai funcoes da comanda e adiciona testes  
2 files changed, 61 insertions(+)
```

Antes de sair da sala

checklist

- 0 `python -m pytest` responde com todas as linhas verdes.
- 1 Nenhuma das suas funções tem `print` dentro.
- 2 O defeito da Aula 3 está corrigido e existe um teste que o cobre.
- 3 `git status` responde: `nothing to commit, working tree clean`.
- 4 A pasta `aula04` aparece no GitHub, sem `__pycache__` nem `.pytest_cache`.

O que ficou de hoje

Ponto	Regra prática
for e while	for quando você sabe a quantidade, while quando depende de uma condição
Percorrer	Nunca altere a coleção que está sendo percorrida. Monte outra
return	Imprimir mostra e some. Devolver entrega para quem chamou
Valor padrão	Só o que não muda. Lista e dicionário, nunca
Escopo	Receba por parâmetro e devolva pelo return. Não dependa de variável de fora
dunder main	Separa rodar o arquivo de importar o arquivo
Teste	É a regra de negócio escrita de forma que o computador confere sozinho

Referências

Python Docs, Controle de fluxo, em português

<https://docs.python.org/pt-br/3/tutorial/controlflow.html>

Python Docs, Módulos, em português

<https://docs.python.org/pt-br/3/tutorial/modules.html>

Python Docs, FAQ: por que valores padrão são compartilhados entre chamadas

<https://docs.python.org/3/faq/programming.html#why-are-default-values-shared-between-objects>

pytest, guia de primeiros passos

<https://docs.pytest.org/en/stable/getting-started.html>

Real Python, Defining your own Python function

<https://realpython.com/defining-your-own-python-function/>

Real Python, What does if `__name__ == "__main__"` do

<https://realpython.com/if-name-main-python/>

Martin Fowler, Refactoring: Extract Function

<https://refactoring.com/catalog/extractFunction.html>

RAMALHO, Luciano. Python Fluente. 2. ed. Novatec, 2023. Capítulo 7

<https://novatec.com.br/livros/python-fluente-2ed/>



Função sem teste é promessa. Com teste, é contrato.

Entrega: pasta aula04 com `comanda.py` e `test_comanda.py` da sua equipe, suíte verde, com push feito, até domingo às 23h59.

Próxima aula: POO I: as suas funções viram classes, atributos e métodos.